

Algorithms Final Examination

Take-home (40%) Due: 2026/06/24

Contents

1	Question 1:	2
2	Question 2:	3
3	Question 3:	11
4	Question 4:	15

1 Question 1:

Question

Design a dynamic programming algorithm to solve the World Series Odds problem and use a table to show the result.

World Series Odds: Two teams, Team A and Team B, play a best-of-seven series (first to win 4 games wins). Let $P(i, j)$ denote the probability that A wins the series given that A still needs i more wins and B still needs j more wins.

Recurrence:

$$P(i, j) = \begin{cases} 1 & \text{if } i = 0, j > 0 \\ 0 & \text{if } i > 0, j = 0 \\ p \cdot P(i - 1, j) + (1 - p) \cdot P(i, j - 1) & \text{otherwise} \end{cases}$$

Assumption: $p = 0.5$ (both teams equally competent).

演算法設計思路

- **Optimal Substructure:** $P(i, j)$ 完全由 $P(i - 1, j)$ 與 $P(i, j - 1)$ 決定，子問題之解可直接組合為原問題之解。
- **Overlapping Subproblems:** 不同的 (i, j) 路徑會重複用到相同的子問題結果，因此以二維陣列記憶化 (memoization) 可避免重複計算。

計算順序：以 bottom-up 方式，從 $i = 0$ 或 $j = 0$ 的 base case 出發，依序填表至 $P(4, 4)$ (World Series 的起始狀態)。

Pseudocode: World-Series-DP

Input: 每局 A 獲勝機率 p ；系列賽先贏 n 局獲勝

Output: A 最終獲勝的機率 $P(n, n)$

```
/* 初始化 base cases */
for j ← 1 to n do
    |  $P[0][j] \leftarrow 1$  ▷ A 已達標，B 尚未，A 必勝
end
for i ← 1 to n do
    |  $P[i][0] \leftarrow 0$  ▷ B 已達標，A 尚未，A 必敗
end
/* Bottom-up 填表 */
for i ← 1 to n do
    for j ← 1 to n do
        |  $P[i][j] \leftarrow p \cdot P[i - 1][j] + (1 - p) \cdot P[i][j - 1]$ 
    end
end
return  $P[n][n]$ 
```

Algorithm 1: WORLD-SERIES-DP(p, n)

填表過程 ($p = 0.5$, $n = 4$)

以下逐格計算 $P(i, j)$ (A 需 i 勝、B 需 j 勝時 A 的獲勝機率)：

Base cases： $P(0, j) = 1$ ($j > 0$)， $P(i, 0) = 0$ ($i > 0$)。

遞推示例：

$$P(1, 1) = 0.5 \cdot P(0, 1) + 0.5 \cdot P(1, 0) = 0.5 \cdot 1 + 0.5 \cdot 0 = 0.5$$

$$P(2, 2) = 0.5 \cdot P(1, 2) + 0.5 \cdot P(2, 1) = 0.5 \cdot 0.75 + 0.5 \cdot 0.25 = 0.5$$

$P(i, j)$		j (B 還需幾勝)				
		$j = 0$	$j = 1$	$j = 2$	$j = 3$	$j = 4$
i (A 還需幾勝)	$i = 0$	—	1	1	1	1
	$i = 1$	0	0.5000	0.7500	0.8750	0.9375
	$i = 2$	0	0.2500	0.5000	0.6875	0.8125
	$i = 3$	0	0.1250	0.3125	0.5000	0.6563
	$i = 4$	0	0.0625	0.1875	0.3438	0.5000

結果

$P(4, 4) = 0.5$ 。

由於雙方實力相當 ($p = 0.5$)，A 贏得七戰四勝系列賽的機率恰為 50%，符合對稱性。計算時間複雜度為 $\Theta(n^2)$ ，空間複雜度亦為 $\Theta(n^2)$ 。

2 Question 2:

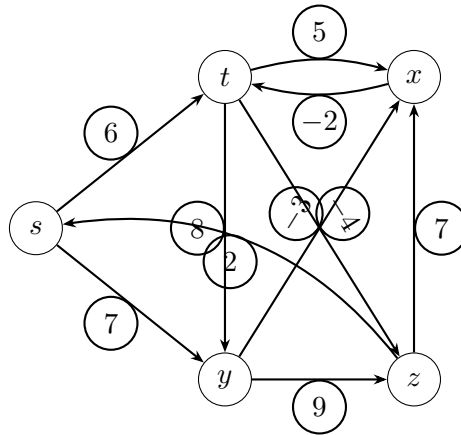
Question

Consider the directed graph in CLRS Figure 24.4 with source vertex s . We implement the modified Bellman-Ford algorithm with the **early termination flag**. The edges are processed in this order each pass:

(s, y) , (y, x) , (x, t) , (t, z) , (t, y) , (x, t) , (y, z) , (z, x) , (z, s) , (s, t)

- Design and trace the modified algorithm: show $v.d$ and $v.\pi$ for all $v \in \{s, t, x, y, z\}$ after each pass until termination.
- State the total number of passes before early termination.

圖結構



(a) 修改 Bellman-Ford 演算法

以課堂原始 pseudocode 為基礎，在第 2-4 行的主迴圈中加入早停旗標。

Initialize-Single-Source 與 Relax (不修改)

```

for each vertex  $v \in V[G]$  do
    | do  $d[v] \leftarrow \infty$ 
    |    $\pi[v] \leftarrow \text{NIL}$ 
end
 $d[s] \leftarrow 0$ 

```

Algorithm 2: INITIALIZE-SINGLE-SOURCE(G, s)

```

if  $d[v] > d[u] + w(u, v)$  then
    | then  $d[v] \leftarrow d[u] + w(u, v)$ 
    |    $\pi[v] \leftarrow u$ 
end

```

Algorithm 3: RELAX(u, v, w)

Bellman-Ford (加入 Early Termination Flag, 以 紅色 標示修改處)

```

INITIALIZE-SINGLE-SOURCE( $G, s$ )
for  $i \leftarrow 1$  to  $|V[G]| - 1$  do
    do  $updated \leftarrow \text{FALSE}$  ▷ 新增：初始化早停旗標

    for each edge  $(u, v) \in E[G]$  do
        do RELAX( $u, v, w$ )
            if  $d[v]$  was updated in Relax then  $updated \leftarrow \text{TRUE}$  ▷ 新增：標記有鬆弛發生
        end
    end
    if not  $updated$  then return TRUE ▷ 新增：本 pass 無更新，提前終止
end
for each edge  $(u, v) \in E[G]$  do
    do if  $d[v] > d[u] + w(u, v)$ 
        then return FALSE
    end
end
return TRUE

```

Algorithm 4: BELLMAN-FORD(G, w, s)

關鍵思路：原始 Bellman-Ford 固定執行 $|V| - 1$ 個 pass (第 2–4 行)。修改版在每個 pass 開始前將旗標 $updated$ 設為 FALSE；若整個 pass 結束後旗標仍為 FALSE (沒有任何邊被成功鬆弛)，代表最短路徑已全部收斂，可直接提前回傳 TRUE，因為後續 pass 也不可能再改變任何 $d[v]$ 。

逐 pass 追蹤

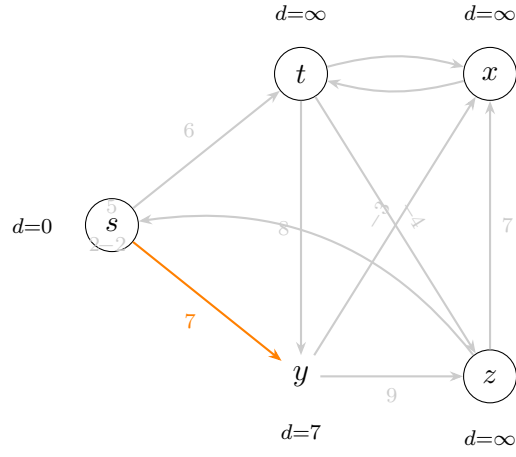
初始狀態 (Pass 0)：

	s	t	x	y	z
$v.d$	0	∞	∞	∞	∞
$v.\pi$	NIL	NIL	NIL	NIL	NIL

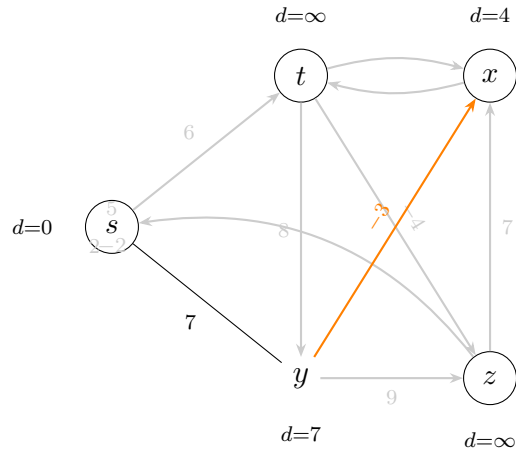
Pass 1 詳細執行：

Pass 1 逐步狀態圖

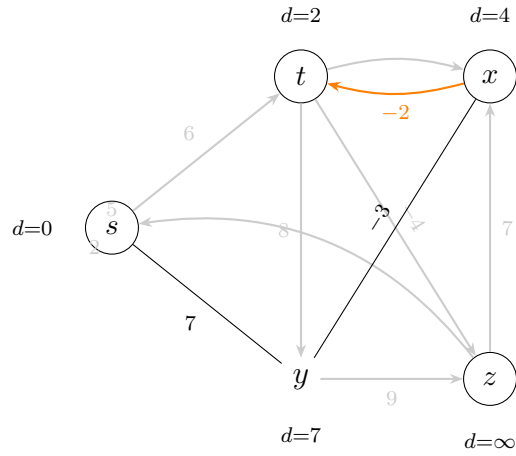
步驟 1：($s \rightarrow y, w = 7$) $y.d : \infty \rightarrow 7, y.\pi = s$



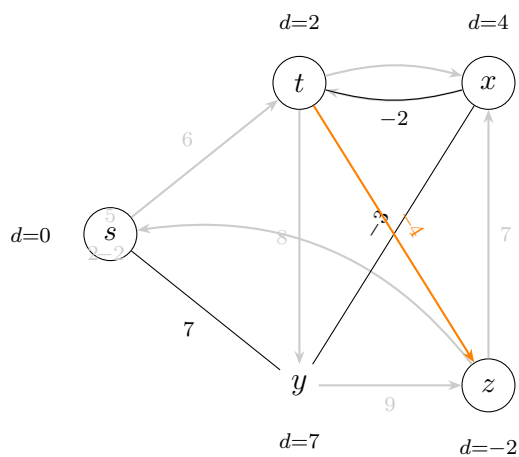
步驟 2 : $(y \rightarrow x, w = -3)$ $x.d : \infty \rightarrow 4, x.\pi = y$



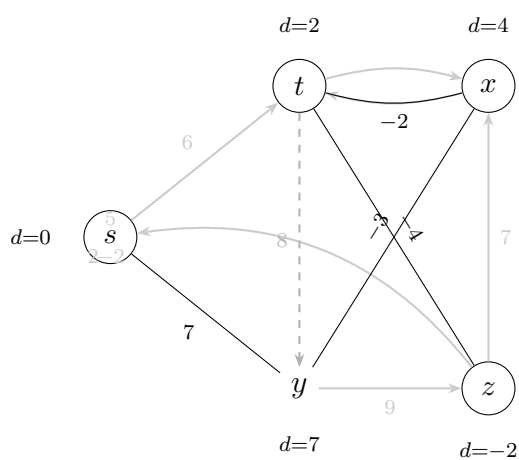
步驟 3 : $(x \rightarrow t, w = -2)$ $t.d : \infty \rightarrow 2, t.\pi = x$



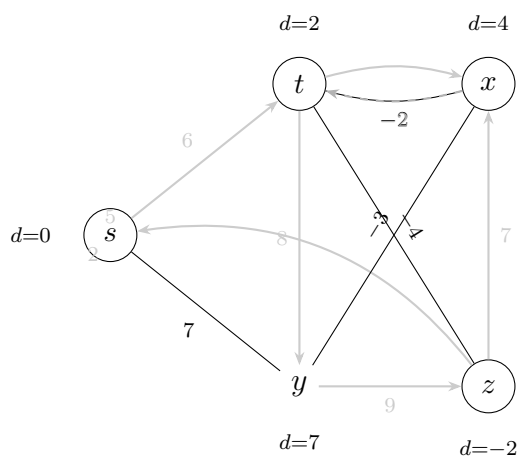
步驟 4 : $(t \rightarrow z, w = -4)$ $z.d : \infty \rightarrow -2, z.\pi = t$



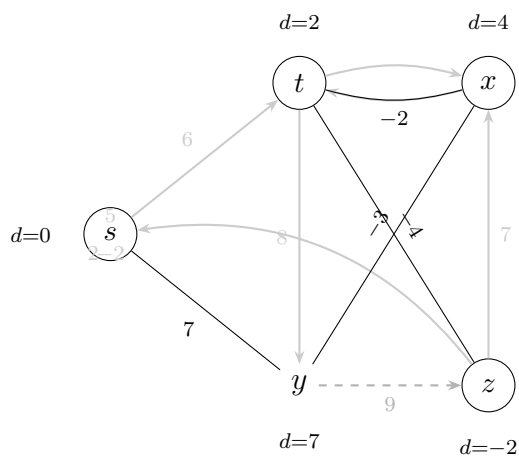
步驟 5 : $(t \rightarrow y, w = 8)$ $2 + 8 = 10 > 7$, 無更新



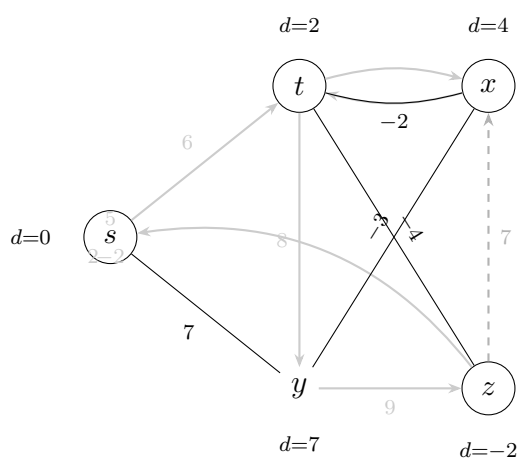
步驟 6 : $(x \rightarrow t, w = -2)$ $4 + (-2) = 2 \not< 2$, 無更新



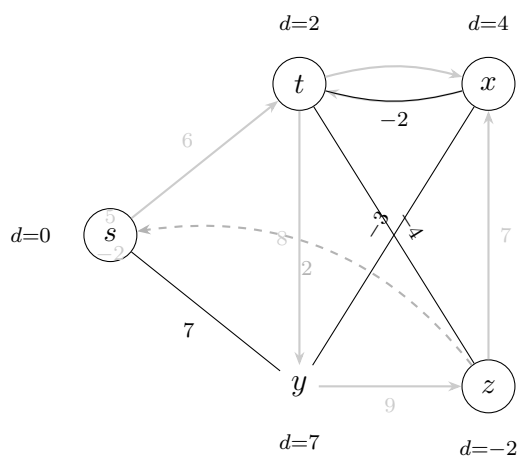
步驟 7 : $(y \rightarrow z, w = 9)$ $7 + 9 = 16 > -2$, 無更新



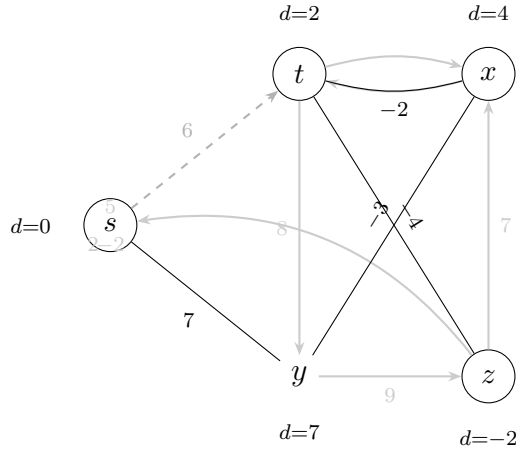
步驟 8 : $(z \rightarrow x, w = 7)$ $-2 + 7 = 5 > 4$, 無更新



步驟 9 : $(z \rightarrow s, w = 2)$ $-2 + 2 = 0 \not> 0$, 無更新



步驟 10 : $(s \rightarrow t, w = 6)$ $0 + 6 = 6 > 2$, 無更新。Pass 1 結束。



updated = true (共 4 次鬆弛)，繼續執行 Pass 2。

步驟	邊	操作	$s.d$	$t.d$	$x.d$	$y.d$	$z.d$
1	$(s \rightarrow y, w = 7)$	$y.d : \infty \rightarrow 7, y.\pi = s$	0	∞	∞	7	∞
2	$(y \rightarrow x, w = -3)$	$x.d : \infty \rightarrow 4, x.\pi = y$	0	∞	4	7	∞
3	$(x \rightarrow t, w = -2)$	$t.d : \infty \rightarrow 2, t.\pi = x$	0	2	4	7	∞
4	$(t \rightarrow z, w = -4)$	$z.d : \infty \rightarrow -2, z.\pi = t$	0	2	4	7	-2
5	$(t \rightarrow y, w = 8)$	$2 + 8 = 10 > 7$ ，無更新	0	2	4	7	-2
6	$(x \rightarrow t, w = -2)$	$4 + (-2) = 2 \not< 2$ ，無更新	0	2	4	7	-2
7	$(y \rightarrow z, w = 9)$	$7 + 9 = 16 > -2$ ，無更新	0	2	4	7	-2
8	$(z \rightarrow x, w = 7)$	$-2 + 7 = 5 > 4$ ，無更新	0	2	4	7	-2
9	$(z \rightarrow s, w = 2)$	$-2 + 2 = 0 \not< 0$ ，無更新	0	2	4	7	-2
10	$(s \rightarrow t, w = 6)$	$0 + 6 = 6 > 2$ ，無更新	0	2	4	7	-2

Pass 1 結束後，updated = true (發生了 4 次鬆弛)，繼續執行。

	s	t	x	y	z
$v.d$	0	2	4	7	-2
$v.\pi$	NIL	x	y	s	t

Pass 2：對所有 10 條邊逐一檢查，發現每條邊均無法進一步鬆弛 (所有 $v.d$ 值不再改變)。
updated = false，觸發 **early termination**，演算法終止。

	s	t	x	y	z
$v.d$	0	2	4	7	-2
$v.\pi$	NIL	x	y	s	t

(b) 總 Pass 次數

答案

演算法共執行 **2 個 pass**：Pass 1 完成所有最短路徑鬆弛；Pass 2 無任何更新，Early termination flag 觸發，提前結束。標準 Bellman-Ford 需執行 $|V| - 1 = 4$ 個 pass，修改版節省了 2 個 pass。

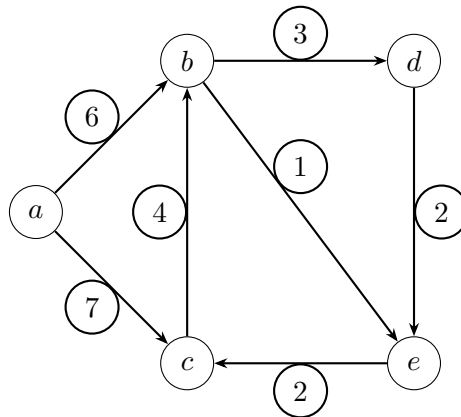
3 Question 3:

Question

Show the operation of the **Dijkstra's algorithm** on the following directed graph, using vertex “a” as the source. In each pass, relax edges, then show d and π . What is the running time of the algorithm?

(Analyze the algorithm in detail. Note: RELAX needs $\lg V$ to maintain the Q .)

圖結構



演算法步驟追蹤

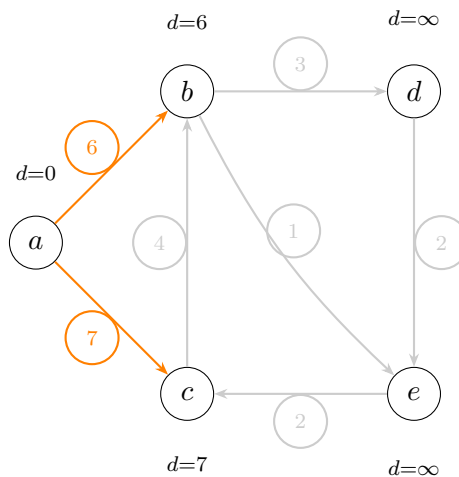
初始化： $a.d = 0$ ，其餘頂點 $d = \infty$ ，所有 $\pi = \text{NIL}$ 。 $Q = \{a : 0, b : \infty, c : \infty, d : \infty, e : \infty\}$ ， $S = \{\}$ 。

Step 1：Extract-Min $\rightarrow$ 取出 a ($a.d = 0$)，加入 S 。

鬆弛 a 的所有出邊：

- RELAX($a \rightarrow b$, $w = 6$) : $b.d = 0 + 6 = 6$, $b.\pi = a$
- RELAX($a \rightarrow c$, $w = 7$) : $c.d = 0 + 7 = 7$, $c.\pi = a$

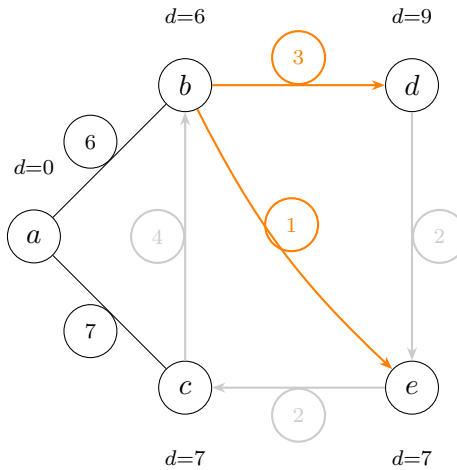
$Q = \{b : 6, c : 7, d : \infty, e : \infty\}$



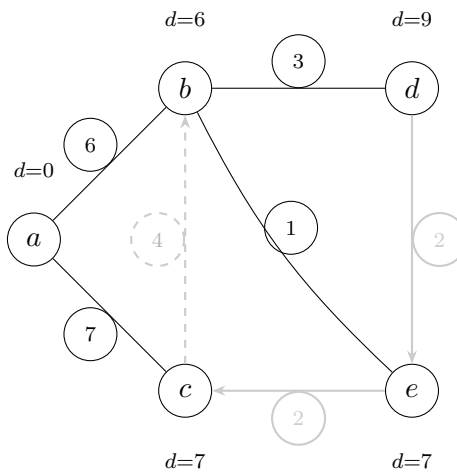
Step 2 : Extract-Min → 取出 b ($b.d = 6$)，加入 S 。
 鬆弛 b 的所有出邊：

- $\text{RELAX}(b \rightarrow d, w = 3) : d.d = 6 + 3 = 9, d.\pi = b$
- $\text{RELAX}(b \rightarrow e, w = 1) : e.d = 6 + 1 = 7, e.\pi = b$

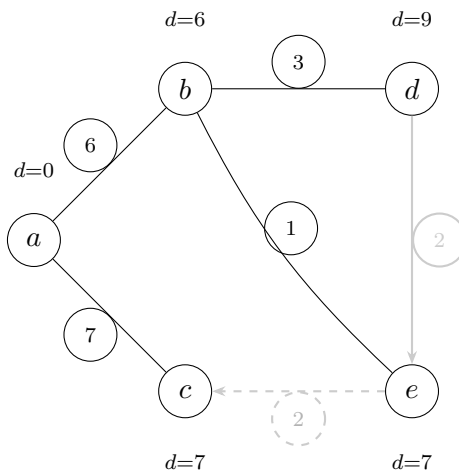
$Q = \{c : 7, e : 7, d : 9\}$



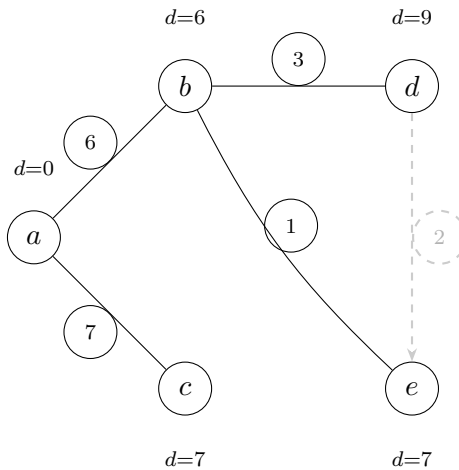
Step 3 : Extract-Min → 取出 c ($c.d = 7$ ，字母序較小)，加入 S 。
 鬆弛 c 的出邊： $\text{RELAX}(c \rightarrow b) : b \in S$ ，略過。無更新。 $Q = \{e : 7, d : 9\}$



Step 4 : Extract-Min → 取出 e ($e.d = 7$)，加入 S 。
 鬆弛 e 的出邊： $\text{RELAX}(e \rightarrow c) : c \in S$ ，略過。無更新。 $Q = \{d : 9\}$



Step 5 : Extract-Min $\rightarrow$ 取出 d ($d.d = 9$)，加入 S 。
 鬆弛 d 的出邊： $\text{RELAX}(d \rightarrow e) : e \in S$ ，略過。 $Q = \{\}$ ，演算法結束。



■ 最短路徑樹完成： $d[b]=6$, $d[c]=7$, $d[d]=9$, $d[e]=7$

完整追蹤表（各 step 後的 d 與 π ）：

Step	取出頂點	a		b		c		d		e	
		d	π	d	π	d	π	d	π	d	π
Init	—	0	—	∞	—	∞	—	∞	—	∞	—
1	a ($d=0$)	S	—	6	a	7	a	∞	—	∞	—
2	b ($d=6$)	S	—	S	—	7	a	9	b	7	b
3	c ($d=7$)	S	—	S	—	S	—	9	b	7	b
4	e ($d=7$)	S	—	S	—	S	—	9	b	S	—
5	d ($d=9$)	S	—	S	—	S	—	S	—	S	—

執行時間分析

以 Binary Min-Heap（二元最小堆積）實作優先佇列 Q ：

操作	每次時間	呼叫次數	總時間
BUILD-MIN-HEAP (初始化 Q)	$O(V)$	1 次	$O(V)$
EXTRACT-MIN (每個頂點取出一次)	$O(\lg V)$	V 次	$O(V \lg V)$
DECREASE-KEY (每次 RELAX 成功時)	$O(\lg V)$	最多 E 次	$O(E \lg V)$
總計			$O((V + E) \lg V)$

執行時間

對本圖 ($|V| = 5, |E| = 7$) :

$$T = O((V + E) \lg V) = O((5 + 7) \lg 5) = O(12 \times 2.32) \approx O(28)$$

一般情況下，使用 binary heap 的 Dijkstra 執行時間為 $O((V + E) \lg V)$ ，若改用 Fibonacci Heap 則可達 $O(V \lg V + E)$ 。

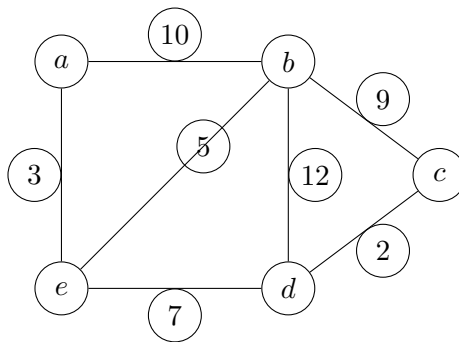
4 Question 4:

Question

As the following graph shown,

- (a) Use *Prim's* algorithm to construct the **minimum spanning tree**. (Show your work step by step where vertex *a* is assigned as *r*)
- (b) What is the running time of the algorithm? (Analyze the algorithm in detail.)

圖結構



(a) Prim's 演算法逐步執行

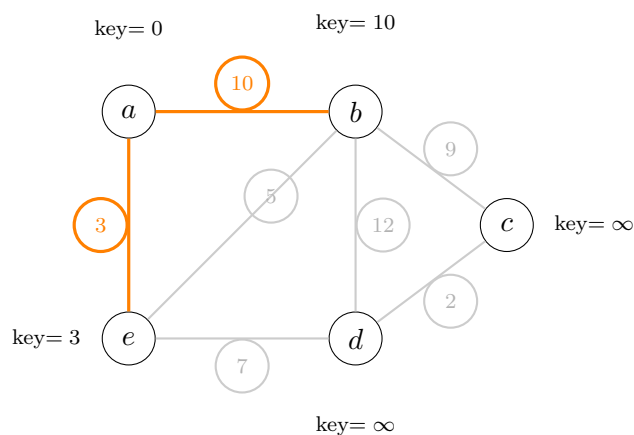
初始化：所有頂點的 $\text{key} = \infty$, $\pi = \text{NIL}$; 令 $a.\text{key} = 0$ $\circ Q = \{a : 0, b : \infty, c : \infty, d : \infty, e : \infty\}$, $\text{MST} = \{\}$ $\circ$

Step 1 : Extract-Min $\rightarrow$ 取出 a ($\text{key} = 0$) $\circ$

更新 a 的鄰居：

- $b : \min(\infty, 10) = 10$, $b.\text{key} = 10$, $b.\pi = a$
- $e : \min(\infty, 3) = 3$, $e.\text{key} = 3$, $e.\pi = a$

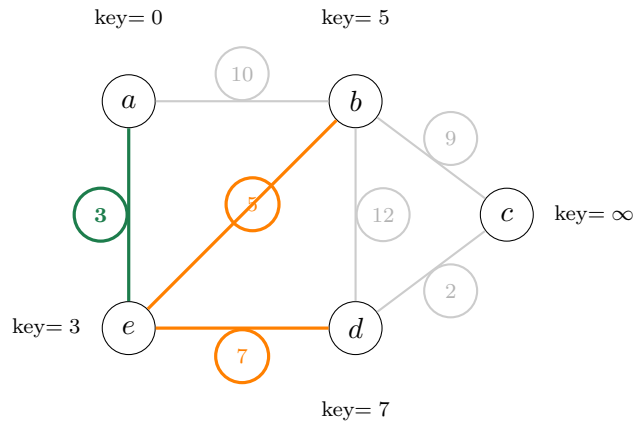
$Q = \{e : 3, b : 10, c : \infty, d : \infty\}$



Step 2 : Extract-Min $\rightarrow$ 取出 e ($\text{key} = 3$) $\circ$ 將邊 (a, e) 加入 MST $\circ$
更新 e 的鄰居 (不在 MST 中者)：

- $b : \min(10, 5) = 5$, $b.key = 5$, $b.\pi = e$ (更新 !)
- $d : \min(\infty, 7) = 7$, $d.key = 7$, $d.\pi = e$

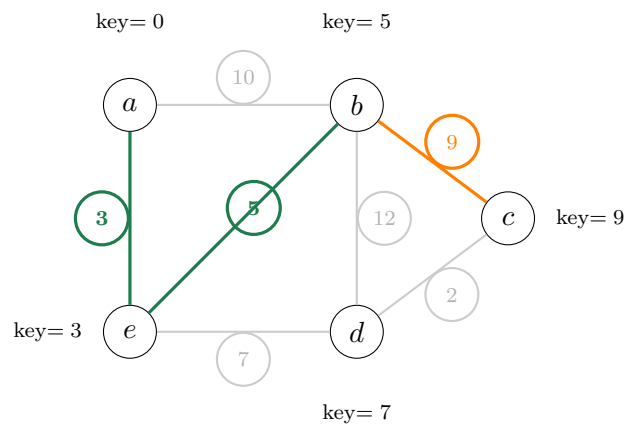
$$Q = \{b : 5, d : 7, c : \infty\}$$



Step 3 : Extract-Min → 取出 b ($key=5$)。將邊 (e, b) 加入 MST。更新 b 的鄰居：

- $c : \min(\infty, 9) = 9$, $c.key = 9$, $c.\pi = b$
- $d : \min(7, 12) = 7$, 不更新

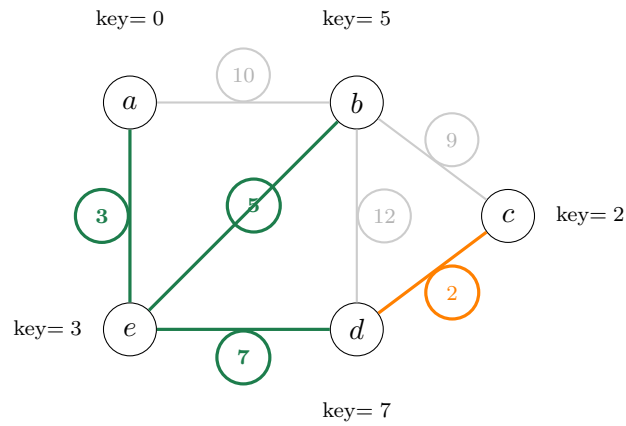
$$Q = \{d : 7, c : 9\}$$



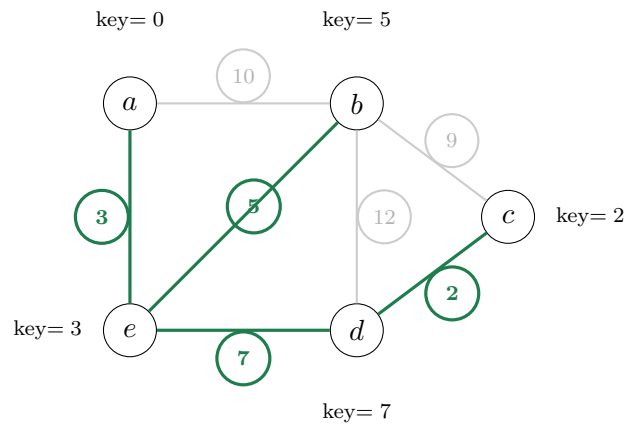
Step 4 : Extract-Min → 取出 d ($key=7$)。將邊 (e, d) 加入 MST。更新 d 的鄰居：

- $c : \min(9, 2) = 2$, $c.key = 2$, $c.\pi = d$ (更新 !)

$$Q = \{c : 2\}$$



Step 5 : Extract-Min → 取出 c ($\text{key}=2$)。將邊 (d, c) 加入 MST。
 $Q = \{\}$ ，演算法結束。



■ MST 完成，總權重 = $3 + 5 + 7 + 2 = 17$

■ MST 邊（綠色粗線），總權重 = 17

(b) 執行時間分析

以 Binary Min-Heap 實作優先佇列 Q ：

操作	每次時間	呼叫次數	總時間
BUILD-MIN-HEAP（初始化）	$O(V)$	1 次	$O(V)$
EXTRACT-MIN（每個頂點恰好一次）	$O(\lg V)$	V 次	$O(V \lg V)$
DECREASE-KEY（每條邊最多觸發一次）	$O(\lg V)$	最多 E 次	$O(E \lg V)$
總計	$O((V + E) \lg V)$		

執行時間

對本圖 ($|V| = 5, |E| = 7$) :

$$T = O((V + E) \lg V) = O((5 + 7) \lg 5) \approx O(28)$$

對稠密圖 ($E = O(V^2)$) : $O(V^2 \lg V)$ 。對稀疏圖 ($E = O(V)$) : $O(V \lg V)$ 。

若以 **Fibonacci Heap** 實作，可達 $O(E + V \lg V)$ ，但實作複雜，常數因子較大，實務上仍常用 binary heap 。